

Project Report: Currency Converter

Project Title: VITyarthi Project - Currency Converter CLI

Student Name: GRACIOUS SHOBY K.

University: VIT Bhopal University

Program: B.Tech Computer Science (1st Year, 2026-2027)

GitHub Repository: <https://github.com/gracious26bcy10096/VITyarthi-project.git>

1. Introduction

The Currency Converter is a lightweight command-line interface (CLI) tool designed in Python. It empowers users to quickly and accurately calculate conversions between 30 different global fiat currencies by utilizing an internal repository of fixed exchange rates. The system emphasizes ease of use and rapid execution within standard terminal environments.

2. Problem Statement

With the increasing globalization of commerce and travel, individuals frequently need to convert between multiple currency pairs. Maintaining a comprehensive NxN matrix of exchange rates for 30 currencies is inefficient. This project addresses the need for a simplified, low-overhead calculation model by mapping all currencies to a common base (USD), thereby standardizing and accelerating the conversion process.

3. Functional Requirements

- The system shall display an active selection menu with options to Enter or Exit.
- The system shall display a formatted table of 30 available currency codes.
- The system shall prompt and accept the base currency, target currency, and numerical amount.
- The system shall calculate and display the output rounded to two decimal places.
- The system shall validate inputs and gracefully reject unknown currency codes.

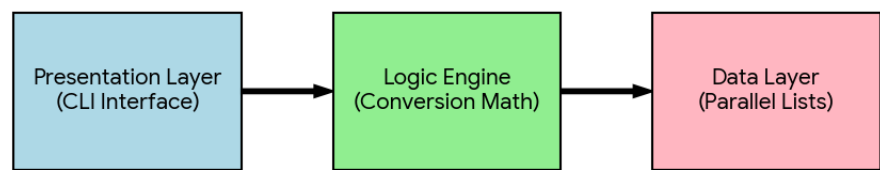
4. Non-functional Requirements

- Performance: Conversions must execute and print instantly via the CLI.
- Usability: The tool must be case-insensitive to user currency inputs.
- Reliability: Infinite looping mechanisms must be safe and user-terminable.
- Portability: The script must run on any OS natively supporting Python 3+

5. System Architecture

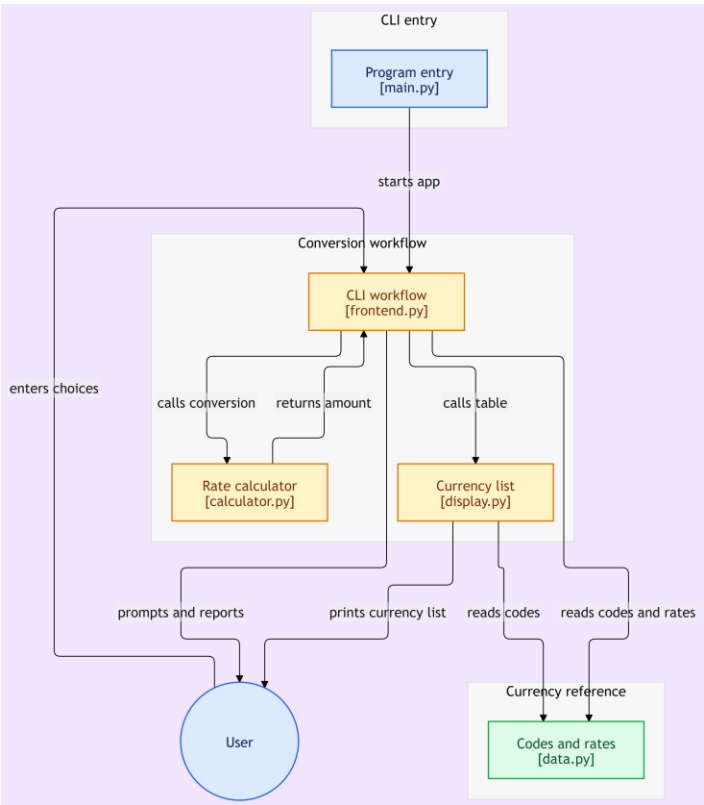
The architecture is divided into three functional layers: a Presentation Layer handling terminal inputs/outputs, a Logic Engine processing the arithmetic, and a Data Layer storing currency definitions.

System Architecture Diagram



6. Design Diagrams

Below is the fundamental Workflow Diagram illustrating the user journey and logical branching:



7. Design Decisions & Rationale

Design Decision	Rationale
Parallel List Data Structure	Using two synchronized lists (`names` and `values`) avoids the need for external database integrations while ensuring $O(N)$ lookup times which are extremely fast for $N=30$.
Two-step conversion via USD pivot	Dividing the input by its native rate normalizes the value to USD, which is then multiplied by the target rate. This avoids hardcoding hundreds of exchange combinations.
.upper() Sanitization	Improves UX by allowing users to type "usd", "Usd", or "USD" interchangeably.

8. Implementation Details

The application is single-threaded and utilizes an infinite `while True:` loop to maintain the program state. The calculation is abstracted into the `convert(amt, value1, value2)` function, which accepts the float amount, normalizes it against `value1`, and scales it against `value2`. Data arrays are statically initialized at the top of the file to ensure they remain globally accessible to the rendering functions like `table(x)`.

9. Screenshots / Results

Simulated Execution Result:

```
-----Currency converter-----
1. Entry
2. Exit

Enter 1 or 2:1

  Available Currencies:
  -----
  ['USD', 'EUR', 'GBP', 'INR', 'JPY', 'CAD', 'AUD', 'AED', 'CHF', 'CNY']
  ['SEK', 'NZD', 'SGD', 'HKD', 'NOK', 'KRW', 'TRY', 'RUB', 'BRL', 'ZAR']
  ['MXN', 'SAR', 'THB', 'IDR', 'MYR', 'PHP', 'DKK', 'PLN', 'CZK', 'HUF']

Currency you have (e.g., USD, INR):INR
Currency you want (e.g., EUR, JPY):EUR
Amount to convert:100

Result: 0.92 EUR

-----Currency converter-----
1. Entry
2. Exit

Enter 1 or 2:|
```

10. Testing Approach

TASK 1: IDENTITY CONVERSION

```
----Currency converter----

1. Entry
2. Exit

Enter 1 or 2:1

Available Currencies:
-----
['USD', 'EUR', 'GBP', 'INR', 'JPY', 'CAD', 'AUD', 'AED', 'CHF', 'CNY']
['SEK', 'NZD', 'SGD', 'HKD', 'NOK', 'KRW', 'TRY', 'RUB', 'BRL', 'ZAR']
['MXN', 'SAR', 'THB', 'IDR', 'MYR', 'PHP', 'DKK', 'PLN', 'CZK', 'HUF']

Currency you have (e.g., USD, INR):USD
Currency you want (e.g., EUR, JPY):INR
Amount to convert:100

Result: 9582.0 INR

----Currency converter----

1. Entry
2. Exit

Enter 1 or 2:
```

RESULT: PASSED

TASK 2: CASE INSENSATIVITY VERIFICATION

```
----Currency converter----

1. Entry
2. Exit

Enter 1 or 2:1

Available Currencies:
-----
['USD', 'EUR', 'GBP', 'INR', 'JPY', 'CAD', 'AUD', 'AED', 'CHF', 'CNY']
['SEK', 'NZD', 'SGD', 'HKD', 'NOK', 'KRW', 'TRY', 'RUB', 'BRL', 'ZAR']
['MXN', 'SAR', 'THB', 'IDR', 'MYR', 'PHP', 'DKK', 'PLN', 'CZK', 'HUF']

Currency you have (e.g., USD, INR):inr
Currency you want (e.g., EUR, JPY):usd
Amount to convert:100

Result: 1.04 USD

----Currency converter----

1. Entry
2. Exit

Enter 1 or 2:|
```

RESULT: PASSED

TASK 3: INVALID SOURCE CURRENCY

```
----Currency converter----

1. Entry
2. Exit

Enter 1 or 2:1

  Available Currencies:
  -----
  ['USD', 'EUR', 'GBP', 'INR', 'JPY', 'CAD', 'AUD', 'AED', 'CHF', 'CNY']
  ['SEK', 'NZD', 'SGD', 'HKD', 'NOK', 'KRW', 'TRY', 'RUB', 'BRL', 'ZAR']
  ['MXN', 'SAR', 'THB', 'IDR', 'MYR', 'PHP', 'DKK', 'PLN', 'CZK', 'HUF']

Currency you have (e.g., USD, INR):abc
Currency you want (e.g., EUR, JPY):INR
Amount to convert:100

Error: We don't have that first currency in our list.

----Currency converter----

1. Entry
2. Exit

Enter 1 or 2:|
```

RESULT: PASSED

TASK 4: INVALID TARGET CURRENCY

```
----Currency converter----

1. Entry
2. Exit

Enter 1 or 2:1

  Available Currencies:
  -----
  ['USD', 'EUR', 'GBP', 'INR', 'JPY', 'CAD', 'AUD', 'AED', 'CHF', 'CNY']
  ['SEK', 'NZD', 'SGD', 'HKD', 'NOK', 'KRW', 'TRY', 'RUB', 'BRL', 'ZAR']
  ['MXN', 'SAR', 'THB', 'IDR', 'MYR', 'PHP', 'DKK', 'PLN', 'CZK', 'HUF']

Currency you have (e.g., USD, INR):INR
Currency you want (e.g., EUR, JPY):abc
Amount to convert:100

Error: We don't have that second currency in our list.

----Currency converter----

1. Entry
2. Exit

Enter 1 or 2:|
```

RESULT: PASSED

TASK 5: INVALID MENU SELECTION

```
----Currency converter----
1. Entry
2. Exit
Enter 1 or 2:7
Invalid input, Please enter 1 or 2.
----Currency converter----
1. Entry
2. Exit
Enter 1 or 2:|
```

RESULT: PASSED

11. Challenges Faced

- Challenge: Floating point arithmetic in Python often yields long decimal tails.
- Solution: Implemented the built-in `round(step2, 2)` to format the final output to a standard financial representation.
- Challenge: Users typing invalid data halting the script.
- Solution: Structured strict `if/elif/else` logic flow rather than raw exceptions to handle non-existent currencies smoothly.

12. Learnings & Key Takeaways

Building this project reinforced foundational computer science concepts, including:

- Array manipulation and parallel indexing mechanics.
- Designing robust CLI user experiences that prevent fatal crashes on bad inputs.
- Organizing Python code into clean, modular functions for reuse.

13. Future Enhancements

- API Integration: Replace static arrays with live JSON data fetched from the ExchangeRate API.
- GUI Migration: Wrap the logic engine into a graphical interface using Tkinter or PyQt.
- Data Persistence: Export user conversion history to a CSV or SQLite database.

14. References

[1] Python Software Foundation. Python 3 Documentation. Available at:

<https://docs.python.org/3/>

[2] Official GitHub CLI Documentation. Available at: <https://docs.github.com/>